AI Usage Note

At the start I uploaded my project source code as a ZIP file so the AI could understand what the project was like and what I already did.

For some of the later prompts though, I did not upload each file again. Instead, I prepared the information I wanted to give to the AI in my own notes/PDF first, then copy-pasted the parts that mattered (like code, UML stuff, table data, or the design document text) straight into the prompt. Because of that, some of the prompts ended up being pretty long and contain big blocks of copied project material.

I mainly used the AI to check spelling and grammar, fix up the wording of some comments, point out possible errors or things that did not match up, and help me format and organize the material I had already made into the final PDF, tables, and UML pages.

If needed, I can also provide the original files/notes that I copied from. They contain the same exact data that was pasted into these prompts.

The following pages are screenshots of the prompts/responses used during the process.

Archive.zip
File

uploaded my codes for phase 2 of project im working on read them only and
i copy pasted what is inside Transaction.h, Transaction.cpp, Session.h and Session.cpp here.
i already wrote all the code so dont change any code at all. dont change logic, methods, variables, signatures, includes, order, validation or anything that affects how it works.
i only want you to check my comments. most of them already say what i mean but some grammar/spelling is not that good and some can probably be a little clearer.
keep the comments close to what i wrote. dont make them super formal and dont add huge explanations.
dont explain anything to me after
just give me back these 4 files:
Transaction.h
Transaction.cpp
Session.h
Session.cpp
with exactly the same code and only the comments improved.

Transaction.h

```
#ifndef TRANSACTION_H
#define TRANSACTION_H

#include <string>
#include "Constants.h"

class Console;
class Session;
class Account;

// Base class for every transaction like login sell buy etc.
// It says if login is needed and what accounts can use it and execute does
// the transaction. Also has helper methods that more then one transaction use.
class Transaction {
protected:
    // Keep console and session so transactions can use them.
```

GPT Medium

```
    // Keep console and session so transactions can use them.
    Console& console;
    Session& session;

    // True if text is empty or END because these cant be used as names.
    static bool isBlankOrEndMarker(const std::string& text);

    // Show the prompt and read a username then find account.
    // Return the account or NULL if input ended or user doesnt exist.
    Account* askForExistingAccount(const std::string& prompt);

    // Ask for amount of credit and read amount bigger than zero.
    // False if input ended or the amount isnt valid.
    bool askForPositiveAmount(Cents& amount);

public:
    // Makes a transaction using the same console and session.
    Transaction(Console& console, Session& session);

    // Need virtual destructor because factory returns Transaction pointer
    // but actual object can be one of the subclasses.
    virtual ~Transaction();

    // Does whole transaction like prompts checks changes and daily line.
    // Every transaction has its own execute.
    virtual void execute() = 0;

    // Most transaction need login but LoginTransaction doesnt.
    virtual bool requiresLogin() const;

    // Normally every account can use transaction.
    // Restricted transaction classes override this.
    virtual bool isPermittedFor(const Account& user) const;
};

#endif
#include "Transaction.h"

#include "Account.h"
#include "Console.h"
#include "FieldFor    h"
#include "Messages.h"
```

+ Ask ChatGPT Medium

```
#include "Session.h"

// Keep the console and session so transaction can use them.
Transaction::Transaction(Console& console, Session& session)
    : console(console), session(session) {
}

// Needed so subclass objects can be deleted using Transaction pointer.
Transaction::~Transaction() {
}

// Most transactions need somebody logged in. Login changes this.
bool Transaction::requiresLogin() const {
    return true;
}

// By default account can use transaction and restricted ones override.
bool Transaction::isPermittedFor(const Account& user) const {
    return true;
}

// Empty text and END cant be used for names.
bool Transaction::isBlankOrEndMarker(const std::string& text) {
    return text.empty() || text == "END";
}

// Ask username then find the account.
// Error is shown here so different transactions dont need same check.
Account* Transaction::askForExistingAccount(
    const std::string& prompt
) {
    std::string username;

    if (!console.ask(prompt, username)) {
        return NULL;
    }

    Account* account = session.getAccounts().find(username);

    if (account == NULL) {
        console.show r(Messages::NO_SUCH_USER);
        return NULL;
```

k ChatGPT

Medium

```
bool Transaction::askForPositiveAmount(Cents& amount) {
    std::string text;

    if (!console.ask(Messages::ENTER_AMOUNT, text)) {
        return false;
    }

    if (!FieldFormat::parseMoney(text, amount) || amount <= 0) {
        console.showError(Messages::INVALID_AMOUNT);
        return false;
    }

    return true;
}
```

Session.h

```
#ifndef SESSION_H
#define SESSION_H

#include <map>
#include <set>
#include <string>
#include <utility>

#include "AccountRegistry.h"
#include "Constants.h"
#include "DailyTransactionLog.h"
#include "GameCatalog.h"

// Three file names that frontend gets from command line.
struct FilePaths {
    std::string accountsFile;
    std::string gamesFile;
    std::string dailyFile;
};

// Everything frontend knows while its running like who is logged in,
// accounts/games, daily transactions, credit added and games bought.
// Transactions get the Session and can read/change these things.
class Session {
private:
    FilePaths filePaths;

    AccountRegistry accounts;
```

atGPT Medium

```
    // Games buyers already bought during this session.
    std::set<std::pair<std::string, std::string> > purchases;

public:
    // Start with nobody logged in and keep file names for later.
    explicit Session(const FilePaths& filePaths);

    // Return three file names.
    const FilePaths& getFilePaths() const;

    // Return accounts being used in this session.
    AccountRegistry& getAccounts();

    // Return games for sale in this session.
    GameCatalog& getGames();

    // Return daily transaction lines collected so far.
    DailyTransactionLog& getLog();

    // True when somebody is currently logged in.
    bool isLoggedIn() const;

    // Start session for this username and clear things from last session.
    void begin(const std::string& username);

    // Return username for whoever is logged in.
    const std::string& getCurrentUsername() const;

    // Return logged in Account or NULL if nobody is logged in.
    Account* getCurrentUser();

    // End session, add final 00 and write daily file.
    // Account gets passed because self delete means its not in registry
anymore.
    bool end(const Account& finalAccountState);

    // Return how much credit already got added to this account this
session.
    Cents getCreditAddedThisSession(
        const std::string& username
    ) const;
```

Medium

```
        const std::string& buyer,
        const std::string& gameName
    ) const;

    // Remember buyer bought this game in this session.
    void rememberPurchase(
        const std::string& buyer,
        const std::string& gameName
    );
};

#endif
Session.cpp
#include "Session.h"

// Nobody is logged in when frontend first starts.
// Keep the file paths because login/logout needs them later.
Session::Session(const FilePaths& filePaths)
    : filePaths(filePaths),
      loggedIn(false) {
}

// Return the three file names.
const FilePaths& Session::getFilePaths() const {
    return filePaths;
}

// Return accounts for this session.
AccountRegistry& Session::getAccounts() {
    return accounts;
}

// Return games for sale for this session.
GameCatalog& Session::getGames() {
    return games;
}

// Return daily transaction lines collected so far.
DailyTransactionLog& Session::getLog() {
    return log;
}
```

tGPT Medium

```
    return currentUsername;
}

// Return account for logged in user or NULL if nobody logged in.
Account* Session::getCurrentUser() {
    if (!loggedIn) {
        return NULL;
    }

    return accounts.find(currentUsername);
}

// finalAccountState gets passed because if admin self deletes,
// current account already isnt in registry but still needed for 00.
bool Session::end(const Account& finalAccountState) {
    // 00 line always has to be last
    log.recordAccountLine(
        "00",
        finalAccountState.getUsername(),
        finalAccountState.getType(),
        finalAccountState.getCredit()
    );

    if (!log.writeToFile(filePaths.dailyFile)) {
        return false;
    }

    loggedIn = false;
    currentUsername.clear();

    return true;
}

// Get how much addcredit already got added to account this session.
// The 1000 limit is for whole session not one command.
Cents Session::getCreditAddedThisSession(
    const std::string& username
) const {
    std::map<std::string, Cents>::const_iterator it =
        creditAddedPerAccount.find(username);

    if (it == creditAddedPerAccount.end()) {
        return 0;
```

hatGPT Medium

```
// that happened while this session is running.
bool Session::hasPurchased(
    const std::string& buyer,
    const std::string& gameName
) const {
    return purchases.find(
        std::make_pair(buyer, gameName)
    ) != purchases.end();
}

// Remember buyer/game pair so another buy can check it later.
void Session::rememberPurchase(
    const std::string& buyer,
    const std::string& gameName
) {
    purchases.insert(
        std::make_pair(buyer, gameName)
    );
}
```

Show less

Transaction.h ↗

Transaction.cpp ↗

Session.h ↗

Session.cpp ↗

Transaction.h
Code

Transaction.cpp
Code

Session.h
Code

1 more

this is for my CSCI 3060U phase 2 design document. i already did the code and project,
dont add any new project info or idea and dont change meaning of anything. only fix spelling
grammar and organize what i give you into clean pdf. dont describe extra stuff yourself.
dont make new requirements or methods or anything.
make first page only for now.

top title:

CSCI 3060U - Phase 2: Front End Design Document

under it:
Digital Games Distribution Service
Arian Vares and Victor Ma
Due October 19, 2026

then put this:
1. Overview
the Front End is a C++ console program. it reads the transactions the user type like
login, logout, create, delete, sell, buy, refund, addcredit, list and listaccounts.
it check them with the user accounts file and available games file when needed and
when user logout it writes the daily transaction file. program run like:

./frontend accounts.txt games.txt daily.txt
later the Back End use the daily transaction file.

for the design we have one class for every transaction. all transaction classes use
Transaction as the base class and TransactionFactory gets the command the user typed
and makes the correct transaction object. Session keeps the things that change while
the program is running like who is login and the session data so we dont need global
variables.

RecordFile reads the files and AccountRegistry and GameCatalog

T Medium

RecordFile reads the files and AccountRegistry and GameCatalog check/load the account
and game data. DailyTransactionLog is for the daily transaction file. all the terminal
input and output goes from Console.

dont add more explanation than this. just clean grammar/spelling and make it readable.
2. How the program runs
make this as numbered list. this is the info:
1. main reads the 3 file names and creates FrontEnd.
2. FrontEnd::run prints "Enter transaction:" and reads a command until input ends.
3. handleCommand ask TransactionFactory to make the transaction for that command.
if command doesnt exist then it gives:
ERROR: invalid transaction

if the transaction needs login but nobody login then:
ERROR: please login first

if someone is login but their account type isnt allowed for that command then:
ERROR: not allowed for this account type

this permission check happens before it asks the other transaction inputs.

4. if everything above is okay then execute() runs. the transaction asks for whatever
inputs it needs and checks the rules. it stops on the first error. if transaction fail
dont record it in daily transaction file.
5. successful transaction update Session if it needs to and if the command needs daily
record it adds the correct line to DailyTransactionLog. login, list and listaccounts
dont make a daily transaction line.

6. logout ends the session. also if an admin delete their own account the session ends.
Session::end adds 00 line and writes daily transaction file. if the input just ends

Medium

3. UML class diagrams
only put the heading and these explanation lines on this page for now.
dont create
the uml pictures yet because i will ask that next.

write:
Figure 1 shows the control flow and the transaction classes.
Figure 2 shows the data, file and helper classes.

Symbols:
+ means public
- means private
means protected
filled diamond means owns
open diamond means holds many
hollow triangle means inherits from
dashed arrow means uses / creates

dont add another uml explanation.
formatting:
make this first page look like normal university design document.
dont make it fancy.
section titles bold.
title at top.
student names and due date under title.
normal font and spacing.
use monospace for ./frontend accounts.txt games.txt daily.txt and
code/class names if it
looks better.
dont add citations or references.
IMPORTANT again:
all the content above is already my project info. dont add or change
project meaning.
only clean my spelling/grammar, organize the page, and make the pdf.give
me only the file and dont explain.

Show less ^

Overview_How_to_run.pdf ↗

Overview_How_to_run.pdf

Ask ChatGPT

Medium

ok now do page 2 and 3 only, the UML part

dont change any class or method or relationship, dont add anythn new. all the UML info already come from my .h files so just draw it clean and organise it. if somethin isnt in the headers dont make it up plz

PAGE 2

on top write:

Figure 1 - FrontEnd, TransactionFactory and the Transaction classes

this page is for the main transaction UML.

put FrontEnd on left side.

FrontEnd has:

-console: Console
-session: Session
-factory: TransactionFactory

+FrontEnd(filePaths, input, output)
+run()
-handleCommand(command)

show that FrontEnd owns Session and TransactionFactory and Console based on the code.

put TransactionFactory near FrontEnd.

TransactionFactory has:

-console: Console&
-session: Session&

+TransactionFactory(console, session)
+create(command): unique_ptr<Transaction>

show FrontEnd ow[redacted]ransactionFactory.

Ask ChatGPT
Medium

<<abstract>>
Transaction

inside it:

#console: Console&
#session: Session&

+Transaction(console, session)
+~Transaction()
+execute() {abstract}
+requiresLogin(): bool
+isPermittedFor(user): bool
#isBlankOrEndMarker(text): bool
#askForExistingAccount(prompt): Account*
#askForPositiveAmount(amount&): bool

show Transaction uses Session and Console.

put Session small box at right and Console small box at lower right, have arrows from Transaction to them and put "uses" on it.

all these transaction classes inherit from Transaction:

LoginTransaction
LogoutTransaction
CreateTransaction
DeleteTransaction
SellTransaction
BuyTransaction
RefundTransaction
AddCreditTransaction
ListTransaction
ListAccountsTransaction

use hollow triangle inheritance arrow going to Transaction.

for each class show methods from the real header.

ListTransaction:

Ask ChatGPT
Medium

+execute()

ListAccountsTransaction:

+ListAccountsTransaction(console, session)
+execute()
+isPermittedFor(user): bool

LoginTransaction:

+LoginTransaction(console, session)
+execute()
+requiresLogin(): bool
-loadAccountsFile(): bool
-loadGamesFile(): bool

LogoutTransaction:

+LogoutTransaction(console, session)
+execute()

CreateTransaction:

+CreateTransaction(console, session)
+execute()
+isPermittedFor(user): bool

DeleteTransaction:

+DeleteTransaction(console, session)
+execute()
+isPermittedFor(user): bool

SellTransaction:

+SellTransaction(console, session)
+execute()
+isPermittedFor(user): bool
-askForGameName(gameName&): bool
-askForPrice(price&): bool

BuyTransaction:

Ask ChatGPT
Medium

+BuyTransaction(console, session)
+execute()
+isPermittedFor(user): bool
-askForGameOnSale(): Game*
-askForSeller(game): Account*
-buyerMayPurchase(game, buyer, seller): bool
-completePurchase(game, buyer, seller)

RefundTransaction:

+RefundTransaction(console, session)
+execute()
+isPermittedFor(user): bool

AddCreditTransaction:

+AddCreditTransaction(console, session)
+execute()
-askForTargetAccount(): Account*
-amountIsAllowed(target, amount): bool

pls check headers before drawing cuz if actual signature has const, reference, pointer etc keep it.

layout should be like:
FrontEnd left
TransactionFactory near center-left
transaction classes mostly vertical in center
Transaction more center-right
Session and Console on far right

dont make lines go thru text if possible.

bottom caption:

Figure 1 - FrontEnd, TransactionFactory and the Transaction classes

PAGE 3

top:

Figure 2 - Session, data, file and helper classes

Ask ChatGPT
Medium

methods:

+Session(filePaths)
+getFilePaths(): FilePaths
+getAccounts(): AccountRegistry&
+getGames(): GameCatalog&
+getLog(): DailyTransactionLog&
+isLoggedIn(): bool
+begin(username)
+getCurrentUsername(): string
+getCurrentUser(): Account*
+end(finalAccountState): bool
+getCreditAddedThisSession(username): Cents
+rememberCreditAdded(username, amount)
+hasPurchased(buyer, gameName): bool
+rememberPurchase(buyer, gameName)

IMPORTANT check actual Session.h first and use exact signature. example if getFilePaths is const FilePaths& then use that exact one, dont copy simplified version if header is different.

put FilePaths above/right of Session.

write:

<<struct>>
FilePaths

+accountsFile: string
+gamesFile: string
+dailyFile: string

show Session owns/contains FilePaths.

put AccountRegistry and GameCatalog to right of Session.

AccountRegistry:

-accounts: vector<Account>

+loadFromFile(path): FileLoadStatus
+find(username): ount*
+add(account)

Ask ChatGPT
Medium

vector<Account>& if thats what code says.

GameCatalog:

-games: vector<Game>

+loadFromFile(path): FileLoadStatus
+findByName(name): Game*
+add(game)
+getAll(): vector<Game>

same, use exact header signature.

show Session owns AccountRegistry and GameCatalog.

show AccountRegistry holds many Account objects and GameCatalog holds many Game objects. put 0..* near those relationship if it fits.

put DailyTransactionLog under/right of Session.

DailyTransactionLog:

-lines: vector<string>

+clear()
+recordAccountLine(code, username, accountType, amount)
+recordSell(gameName, seller, price)
+recordBuy(gameName, seller, buyer, price)
+recordRefund(buyer, seller, amount)
+writeToFile(path): bool

show Session owns DailyTransactionLog.

put Account top right.

Account fields:

-username: string
-accountType: string
-credit: Cents

methods:

+ Ask ChatGPT Medium

-credit: Cents

methods:

+Account()
+Account(username, accountType, credit)
+isValidType(accountType): bool {static}
+parseFileLine(line, account&): bool {static}
+getUsername(): string
+getType(): string
+getCredit(): Cents
+isAdmin(): bool
+canBuy(): bool
+canSell(): bool
+canReceive(amount): bool
+addCredit(amount)
+removeCredit(amount)

put Game lower right.

Game fields:

-name: string
-seller: string
-price: Cents
-listedThisSession: bool

methods:

+Game()
+Game(name, seller, price)
+parseFileLine(line, game&): bool {static}
+getName(): string
+getSeller(): string
+getPrice(): Cents
+wasListedThisSession(): bool
+markListedThisSession()

put RecordFile between Account/Game and registries.

RecordFile:

+readLines(path, lines&): FileLoadStatus {static}

Ask ChatGPT
Medium

methods:

+padRight(text, width): string {static}
+trimRight(text): string {static}
+moneyField(cents, width): string {static}
+moneyDisplay(cents): string {static}
+parseMoney(text, cents&): bool {static}
+parseMoneyField(field, cents&): bool {static}

show dashed use arrows from Account, Game and DailyTransactionLog to FieldFormat only if thats what current code actually does.

for both pages:

white background
simple uml boxes
light header bar is fine
black lines
normal student project style
dont add explanation paragraphs
dont put extra notes
dont add methods not in headers
dont change the project design
make diagrams readable and centered
leave enough margin around page

page 2 should only have Figure 1 UML and caption.
page 3 should only have Figure 2 UML and caption.
rename the file as uml.pdf

Show less

UML.pdf ↗

UML.pdf
PDF

ChatGPT can make mistakes. Check important info.

Ask ChatGPT

Medium

ok now do the rest of the design pdf, from section 4 to the end use the files uploaded earlier when ever was mentioned and give me the final pdf also add uml.pdf and overview part too and give final file.

same thing as before, dont change anything in my code or project meaning. dont make new
requirements or new behavior. basically read the files i say and put the info in the
tables/sections. if something isnt in code or in the info i give below dont guess it.

dont explain stuff to me in the pdf, just make the actual document.

also keep writing simple like student project, not super professional or long.

4. Where the inputs and outputs fit

...

Show more

Phase2_Design_Document.pdf ↗

Phase2_Design_Document.pdf
PDF

ChatGPT can make mistakes. Check important info.

Ask ChatGPT

Medium

runs it.

IMPORTANT for section 9:

if an intention isnt literally written in code, it is ok to make the short intention
from the actual implementation.

but dont add design reasons that arent there.

example:

getUsername()
"Returns the username."

addCredit(amount)
"Adds the amount to the credit."

loadFromFile(path)
"Loads and checks the file data."

so just simple descriptions.

format all pages same style as first pages.

normal heading.
tables light grey/blue header is fine.
no fancy design.
font readable.
dont make tables go outside page.
if table doesnt fit continue on next page.
section 9 can use many pages, thats fine.
keep class/method names in code style if possible.

dont add references/citations.

again dont change project meaning, dont fix code, dont make new methods or errors.
just use my files + ↓ text i gave here and organize it into the pdf.

mark them static if header says static.

Constants / Messages - Victor

use:

Constants.h
Messages.h

these arent classes so just make a small row called:

Constants.h / Messages.h

intention should just say:

Constants has shared limits, widths, command words, account types and transaction codes.
Messages has prompts, confirmation and error text.
Cents is money stored in whole cents.

dont list every single constant unless needed.

main - Victor

use:
main.cpp

make small table:

main
main(argc, argv): int

intention:

Program entry point.

and for method:

checks exactly 3 file names, puts them in FilePaths, makes FrontEnd and runs it.

removeCredit

Game - Victor

use:
Game.h
Game.cpp

include:

Game
constructors
parseFileLine
getName
getSeller
getPrice
wasListedThisSession
markListedThisSession

Console - Victor

use:
Console.h
Console.cpp

include:

Console
constructor
show
showError
readLine
ask

FieldFormat - Victor

use:
FieldFormat.h
FieldFormat.cpp

GameCatalog.cpp

class
loadFromFile
findByName
add
getAll

RecordFile / FileLoadStatus - Arian

use:
RecordFile.h
RecordFile.cpp

include:
FileLoadStatus
RecordFile
readLines

DailyTransactionLog - Arian

use:
DailyTransactionLog.h
DailyTransactionLog.cpp

include:

class
clear
recordAccountLine
recordSell
recordBuy
recordRefund
writeToFile

ListAccountsTransaction - Victor

use:
ListAccountsTransaction.h/.cpp

class
constructor
execute
isPermittedFor

Session / FilePaths - Arian

use:
Session.h
Session.cpp

include FilePaths and Session.

for Session use every public method from header.

things like:

getFilePaths
getAccounts
getGames
getLog
isLoggedIn
begin
getCurrentUsername
getCurrentUser
end
getCreditAddedThisSession
rememberCreditAdded
hasPurchased
rememberPurchase

IMPORTANT use signature from actual header.
dont simplify return types.

class
constructor
execute
isPermittedFor
askForGameName
askForPrice

BuyTransaction - Arian

use:
BuyTransaction.h/.cpp

include:

class
constructor
execute
isPermittedFor
askForGameOnSale
askForSeller
buyerMayPurchase
completePurchase

RefundTransaction - Arian

use:
RefundTransaction.h/.cpp

class
constructor
execute
isPermittedFor

AddCreditTransaction - Arian

use:
AddCreditTransac .h/.cpp

~Transaction()
execute()
requiresLogin()
isPermittedFor(user)
isBlankOrEndMarker(text)
askForExistingAccount(prompt)
askForPositiveAmount(amount)

if static in header show static.
if protected use #.
if abstract show abstract.

LoginTransaction - Arian

use:
LoginTransaction.h
LoginTransaction.cpp

include:

LoginTransaction
constructor
execute
requiresLogin
loadAccountsFile
loadGamesFile

LogoutTransaction - Victor

use:
LogoutTransaction.h
LogoutTransaction.cpp

include class, constructor and execute.

CreateTransaction - Victor

use:
CreateTransaction..

9. Class and method descriptions

this section is longer.

IMPORTANT:
for this section read the .h file first for actual method list and signature.
then use .cpp only to know what that method is doing.

dont invent new intention.
the "intention" column just means give 1 short sentence saying what this method/class
does based on the actual code.

for example if method is getUsername then intention can just say "Returns the username."
dont make fancy explanation.

make for every class a small table with 2 columns:

Class / method
Intention

do these classes in this order:

FrontEnd - Arian

use:
FrontEnd.h
FrontEnd.cpp

rows:

FrontEnd
FrontEnd(filePaths, input, output)
run()
handleCommand(command)

Arian side put:

FrontEnd
TransactionFactory
Transaction
Session (with FilePaths)
RecordFile
AccountRegistry
GameCatalog
DailyTransactionLog
LoginTransaction
SellTransaction
BuyTransaction
RefundTransaction
AddCreditTransaction

UML diagrams
sections 1 to 3

and for section 9 Arian does rows for his classes.

Victor side put:

Constants.h
Messages.h
FieldFormat
Console
Account
Game
LogoutTransaction
CreateTransaction
DeleteTransaction
ListTransaction
ListAccountsTransaction
main.cpp
Makefile

Sections 4 to 6 and 8

and for section 9 Victor does rows for his classes

- program arguments:

if not exactly 3 file names then main prints usage to standard error and stops.

dont make up other assumptions.

7. Not finished or tested yet (left for Phase 3)

make that exact heading.

put this paragraph:

This is the rapid first version. It builds and runs on simple inputs, but we have not
run our 74 requirements tests against it. Anything those tests find (output wording,
edge cases around limits, the deleted-username conflict above, bad input in the middle
of a transaction) will be fixed in Phase 3.

dont add more to it.

8. Team split

make title:

8. Team split

make a 2 column table.

left title:

Arian Vares

right title:

Victor Ma

Front End only writes daily transaction file.

- price 0.00:

sell allows 0.00/free game.

- addcredit:

1000.00 session limit applies to every addcredit including admin adding to another account.

- accounts/games:

read from files at every login.
files themselves arent changed by Front End.

- ownership:

Front End doesnt receive Game Collection File so buy only remembers purchases from current
session.

- deleting user:

doesnt remove games that user listed.

- admin self delete:

session ends right away.
02 delete line then 00 line.
dont print Logout successful.

- seller mismatch:

if typed seller account exists but doesnt match the seller on listing,

Errors:
none besides normal permission handling before transaction

Success:

Active accounts:
then username | type | credit

Daily:
none

under this transaction table add short shared rules paragraph:

money is stored as whole cents.
max credit is 999999.99.
username is case sensitive and max 15 chars.
game name max 25 chars and case sensitive.
game names are unique across sellers.
a game listed in current session cant be bought until later session.
refund takes money from seller.
addcredit max 1000.00 per account each session.

dont add more rules.

6. Design decisions and assumptions

make heading:

6. Design decisions and assumptions

put these as short bullets or short paragraphs.
dont make them long.

- Reusing a deleted username:

client said on Slack a deleted username can be created again in same

everyone that is logged in

Input:
none/stdin none

Prompt:
none

Errors:
please login first
cannot write daily transaction file

Success:
Logout successful.
only when daily file was written

Daily:
00 + daily file written

CREATE ROW

use:

CreateTransaction.cpp
Account.cpp
AccountRegistry.cpp
Messages.h